

AMR 관제(FMS) 연동 현장 I/O 시스템

라즈베리파이 GPIO 시뮬레이션으로 만든
경광등 · 부저 · 버튼과 로봇 작업의 연동

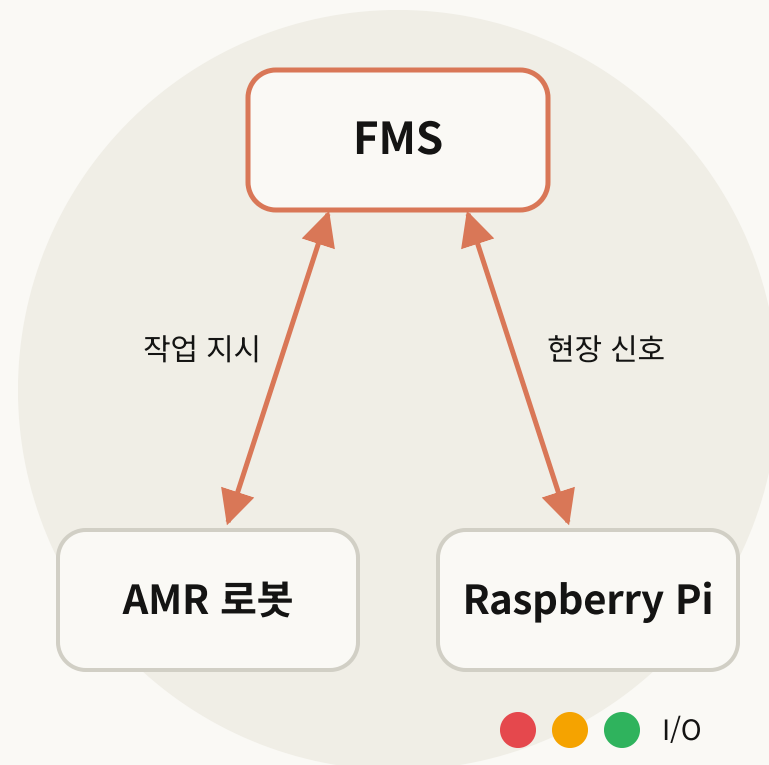
Raspberry Pi GPIO

Flask-SocketIO

MQTT

FastAPI

김대환 2026.10.02



배경 — 로봇과 현장 설비가 따로 움직인다

01

FMS

여러 대의 AMR(자율이동로봇)에 작업을 배정하고, 경로·위치·상태를 관제하는 서버. 관리 대상은 **로봇뿐**이다.

02

단절된 현장 설비

실제 현장에는 경광등, 부저, 확인 버튼 같은 장치가 있다. 지금은 로봇 작업과 **연결되어 있지 않다**.

03

실물 없는 개발

현장 장치는 라즈베리파이 GPIO로 제어할 계획이지만, **개발 시점에 실물 보드가 없다**.

핵심 질문 — **로봇의 작업 흐름 안에 현장 I/O를 어떻게 끼워 넣을 것인가?**

예: "로봇이 적재 위치에 도착하면 경광등을 켜고, 작업자가 버튼을 누를 때까지 기다렸다가 출발"

목표

1

현장 입력 → FMS 판단

버튼 같은 현장 신호를 FMS가 받아, 작업(job) 진행을 멈추거나 이어간다.

2

FMS → 현장 출력

작업 단계 안에서 FMS가 경광등·부저를 직접 켜고 끈다. 라즈베리파이는 스스로 판단하지 않는다.

3

실물 이전 시 코드 수정 0

가상 GPIO로 개발하되, 실물 라즈베리파이에 그대로 올리면 동작해야 한다. 바꾸는 것은 배선 설정 파일뿐.

전체 구조



사용 기술

HARDWARE

하드웨어

Raspberry Pi (GPIO 40핀 헤더) · 경광등 적·황·녹 · 능동 부저 · 푸시 버튼

실물 보드 없이 가상 GPIO로 개발, 실물 연결 시 경광등·부저는 릴레이·트랜지스터 경유

SOFTWARE

소프트웨어

Raspberry Pi OS · Python · Flask-SocketIO · FastAPI · RPi.GPIO (rpi-lgpio) · HTML / JavaScript

mockgpio: RPi.GPIO와 같은 함수로 직접 구현한 가상 GPIO · 로봇은 C++ 시뮬레이터

PROTOCOL

통신

MQTT (Mosquitto) · HTTP REST · Socket.IO / WebSocket

로봇 ↔ FMS는 MQTT, 라즈베리파이 ↔ FMS는 HTTP, 웹 화면은 실시간 푸시

DEPLOY

배포

Google Cloud VM (Linux) · systemd 상시 실행

FMS·시뮬레이터·브로커·로봇을 클라우드에 올리고 브라우저로 IP 접속

IOT

IoT 기술

GPIO 디지털 입출력 · 엣지 인터럽트 감지 · 풀업 · 액티브 로우

센서 상태 업로드 · 원격 출력 제어 · 원격 모니터링

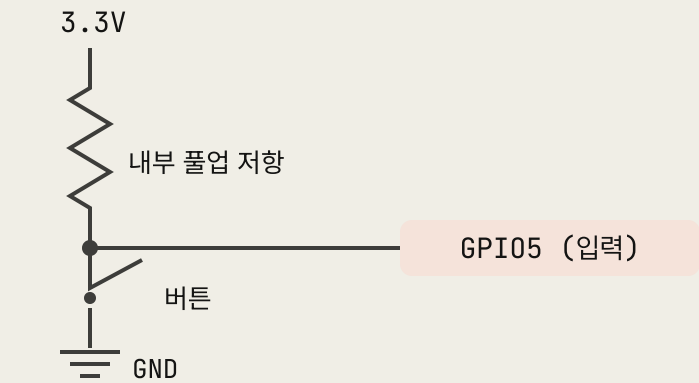
상태는 변경 즉시 + 2초 주기로 FMS에 전송, 6초 넘게 끊기면 신호 없음으로 처리

하드웨어 구성 — 핀 맵과 풀업 버튼

장치	BCM	물리 핀	방향	활성 레벨
● 경광등 적색	17	11	출력	HIGH
● 경광등 황색	27	13	출력	HIGH
● 경광등 녹색	22	15	출력	HIGH
부저 (능동형)	23	16	출력	HIGH
푸시 버튼	5	29	입력	LOW (active-low)

핀 번호와 극성은 `io_config.py` 한 곳에서만 정의한다.
경광등(12/24V)·부저는 GPIO(3.3V)로 직접 구동하지 않고 릴레이/트랜지스터를 거친다.

버튼은 왜 기본이 HIGH인가



안 누름	HIGH	→ 논리 0 · released
누름 (GND 연결)	LOW	→ 논리 1 · pressed

가상 GPIO — 실물과 같은 코드 경로

```
# 실물이 있으면 실물, 없으면 가상 GPIO
try:
    import RPi.GPIO as GPIO
except (ImportError, RuntimeError):
    from mockgpio import GPIO

GPIO.setup(5, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.add_event_detect(5, GPIO.BOTH,
                      callback=_on_edge, bouncetime=20)
```

- RPi.GPIO와 같은 이름 · 시그니처 · 상수 값 (BCM=11 , PUD_UP=22 ...)
- 가상 전용 메서드는 `set_input(pin, level)` 하나 — 센서 입력을 흉내 낸다

실물

버튼을 누름

시뮬레이터

Hold to press

↓ 핀 레벨 HIGH → LOW

엣지 감지 → 콜백 `_on_edge()`

실물: 하드웨어 스레드 · 가상: `set_input`

↓ 여기부터 완전히 같은 코드

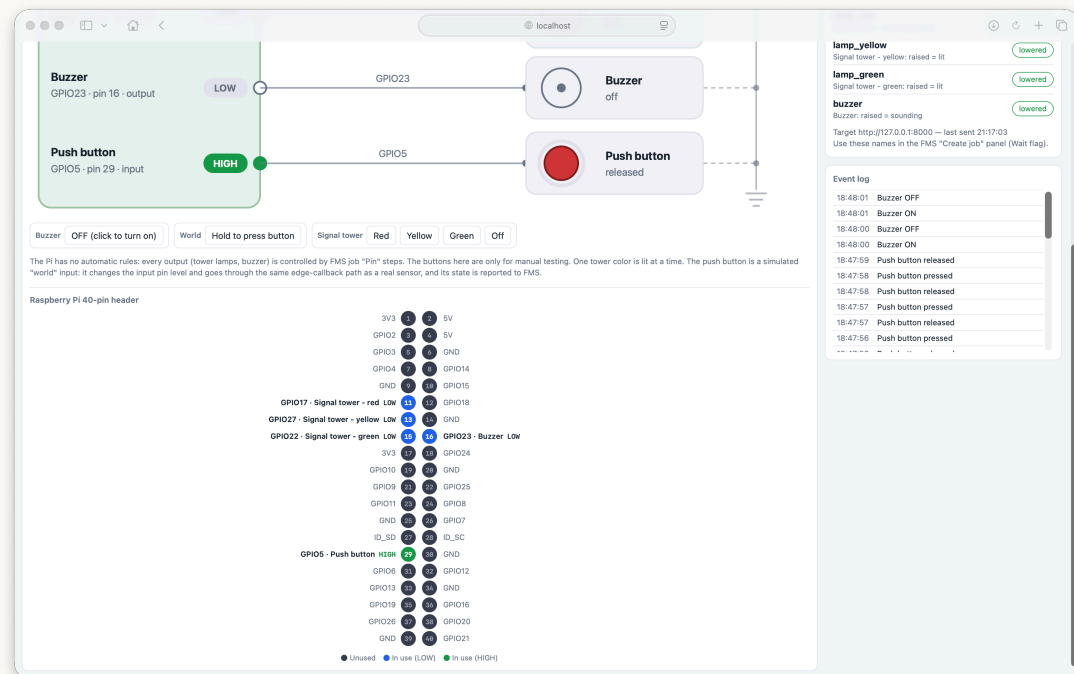
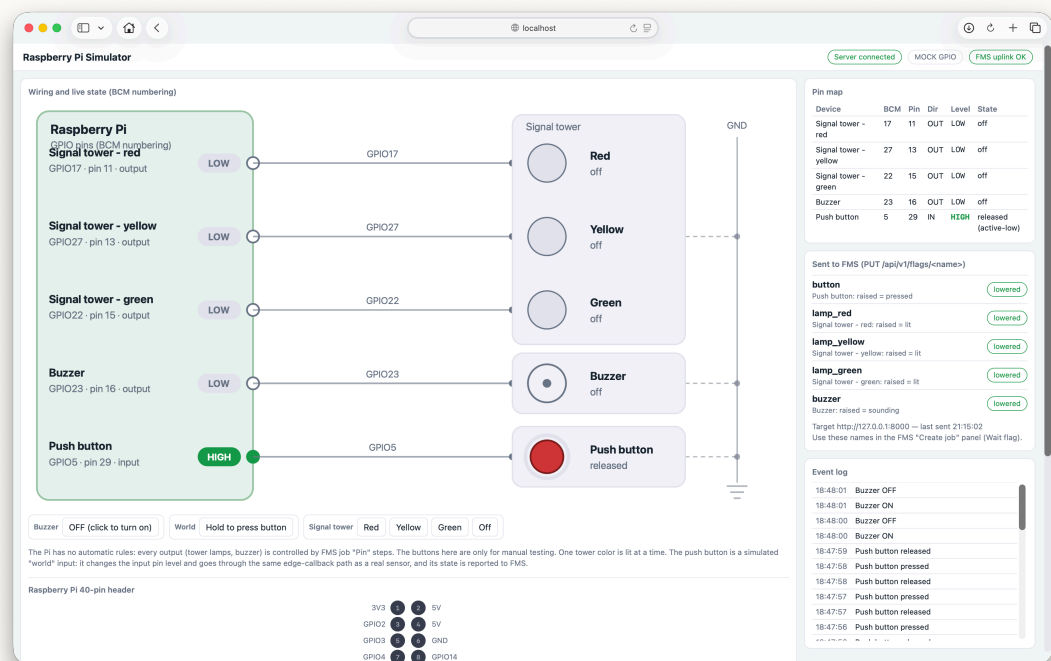
대시보드 갱신

Socket.IO

FMS 업로드

button = true

라즈베리파이 시뮬레이터 화면



1 배선도

GPIO 핀 → 장치 연결, 커진 선은
초록, 공통 GND

2 실시간 레벨

핀마다 HIGH / LOW, 장치 상태
ON · SOUNDING · PRESSED

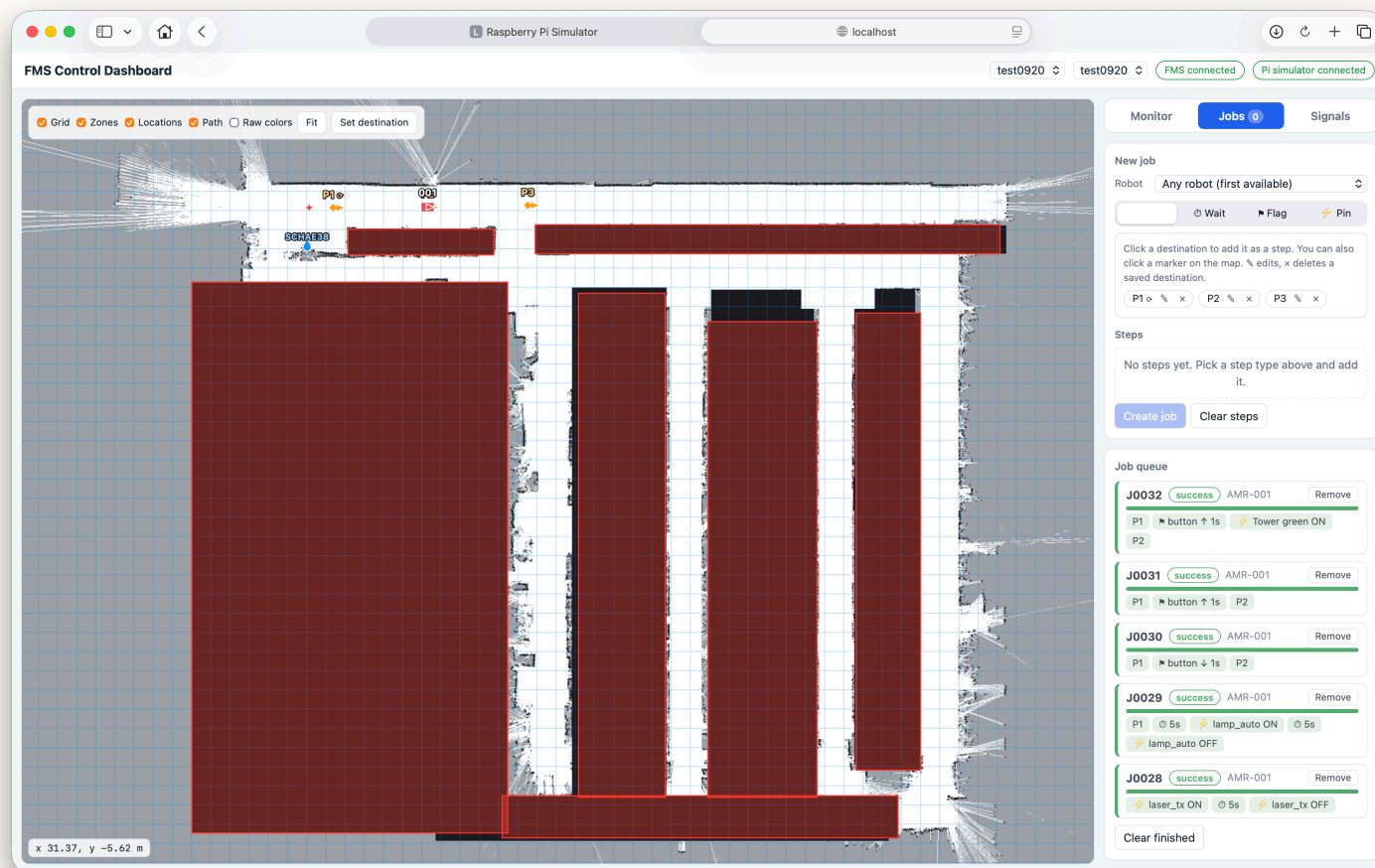
3 40핀 헤더

실제 헤더 배열에서 사용 중인 핀만
강조

4 FMS 업로드 상태

FMS로 보내는 신호, 전송 성공 여부, 이벤트 로그

FMS 대시보드 — 맵 위의 로봇



1 그리드 맵 + 실시간 로봇

로봇 위치·방향을 200ms마다 갱신, 현재 목적지까지 경로만 표시

2 목적지 등록

맵 클릭으로 좌표·방향 저장, 경유 시 방향 유지 옵션

3 Follow

선택한 로봇을 화면 중앙에 고정

4 탭 구성

Monitor · Jobs · Signals

FMS 서버는 기존 관제 서버를 복사해 확장했다 (원본은 수정하지 않음).

Job — 로봇 이동과 현장 I/O를 한 시퀀스로

➔ Move

저장한 목적지로 이동

Move to loc1

⌚ Wait

지정 시간 대기

Wait 2 s

▮ Flag

현장 신호 조건까지 대기

button raised

⚡ Pin

라즈베리파이 출력 제어

Tower yellow ON

Move · Wait 는 기존 기능, **Flag · Pin** 은 현장 I/O 를 위해 추가한 단계

작업 큐

pending → working → success / failed · canceled

- 로봇이 일하는 중에도 생성 가능 — **먼저 만든 순서대로** 배정
- 로봇 지정 또는 **아무 로봇이나** (먼저 한가해진 로봇)
- 대기 단계 중에도 로봇은 그 job에 묶여 다른 job을 받지 않는다

예시 job — 도착 · 신호 · 출발

➔ loc1 ⚡ 황색 ON ▮ button raised ⚡ 부저 ON ⌚ 2s
⚡ 부저 OFF ⚡ 녹색 ON ➔ loc2

loc1에 도착하면 경광등을 켜고, 작업자가 버튼을 누를 때까지 기다렸다가 부저로 알린 뒤 loc2로 출발

신호 연동 설계

원칙 — 판단은 **FMS가**, 라즈베리파이는 **I/O만**. 파이에는 자동 규칙이 없다.

Pi → FMS : 원시 상태 업로드

- 장치 이름 그대로 올린다: `button` `buzzer` `lamp_red` ...
- 값이 바뀌면 즉시, 안 바뀌어도 **2초마다 재전송** (FMS 재시작 복구)
- `raised(1)` = 활성 — 눌림 · 울림 · 점등

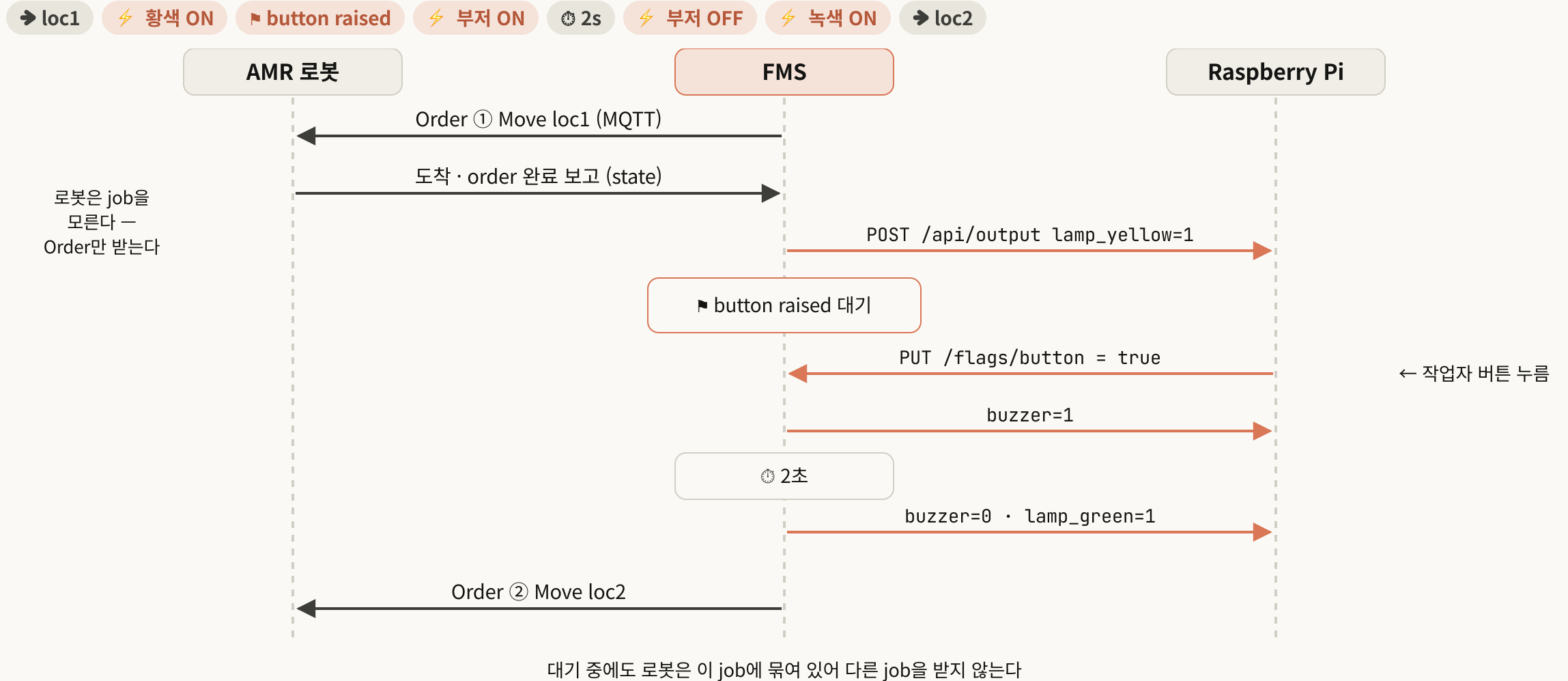
FMS → Pi : 출력 명령

- Pin 단계에서 `POST /api/output` — 실패하면 job 실패
- 경광등은 한 번에 한 색만

Flag 단계의 조건 (FMS에서 정한다)

Until	raised(1) 또는 lowered(0)이 될 때까지
Hold	조건이 N초 동안 끊김 없이 유지돼야 통과 (틈 무시)
Timeout	N초 안에 안 되면 job 실패 / 그냥 진행
신호 만료	6초 넘게 소식이 없으면 조건 불충족 — 파이가 끊겨도 잘못 통과하지 않는다

실행 흐름 — 한 job이 쪼개져 실행되는 과정



시연 및 Q&A

➔ loc1 ⚡ 황색 ON ▶ button raised ⚡ 부저 ON ⌚ 2s ⚡ 부저 OFF
⚡ 녹색 ON ➔ loc2

- 1 **Job 생성**
Jobs 탭에서 단계를 쌓고 Create job → 큐에 들어가 로봇에 배정
- 2 **loc1 도착 → 경광등 황색**
Pi 화면에서 GPIO27이 HIGH로 바뀔, job은 버튼 대기
- 3 **버튼 누름**
Pi 화면의 버튼 → FMS 신호 button = raised
- 4 **부저 2초 → 녹색 → loc2 출발**
GPIO23 · GPIO22 변화, 로봇이 다음 목적지로 이동

화면 배치: 왼쪽 FMS 대시보드, 오른쪽 라즈베리파이 시뮬레이터

Q & A
감사합니다

김대환 · IoT 프로그래밍